

Terraform Infrastructure Protection: Preventing Accidental Resource Deletion

The Problem

While managing infrastructure with Terraform, I had a critical EC2 instance that was important and should not be deleted.

During maintenance, someone accidentally ran `terraform destroy`, which attempted to delete the instance.

I needed a way to make Terraform **block the accidental deletion** of this resource. So Terraform would stop the deletion of this specific resource.

How I Debugged It

I ran: `terraform destroy`

Terraform stopped the operation and showed: **"Error: Instance cannot be destroyed"**

I checked the Terraform configuration and found that the EC2 instance had a lifecycle protection rule.

The Solution

I added the mandatory configuration:

```
lifecycle {  
    prevent_destroy = true  
}
```

This protection can be added to **any Terraform resource that needs protection**. It is not automatically applied to all resources—you must add it to each resource you want to protect.

Here Is How I Fixed It

I added the `prevent_destroy = true` lifecycle rule to the EC2 resource and verified that

Terraform blocked the deletion attempt. With a clear screenshot and doc steps below

Key Takeaway

For critical Terraform-managed resources, `prevent_destroy = true` provides an additional safety layer against accidental deletion.

Problem → Debug → Fix → Protection

Note: This protection applies to Terraform-managed deletion. It does not prevent someone from deleting the resource directly through the AWS Console or AWS CLI.

Has anyone else faced this problem?

How did you fix it?

Step1: Now you can run Terraform destroy; it will not delete the resources or any services

```
Plan: 0 to add, 0 to change, 1 to destroy.
```

Error: Instance cannot be destroyed

on Main.tf line 13:

```
13: resource "aws_instance" "backupballdatabase" {
```

Resource aws_instance.backupballdatabase has lifecycle.prevent_destroy set, but the plan calls for this resource to be destroyed. To avoid this error and continue with the plan, either disable lifecycle.prevent_destroy or reduce the scope of the plan using the -target option.

EC2 > Instances

EC2

Dashboard

AWS Global View

Events

▼ Instances

Instances

Instances (4) Info

Last updated less than a minute ago

Connect

Instance state ▼

Actions ▼

Launch instances ▼

Saved filter sets

Choose filter set ▼

Find Instance by attribute or tag (case-sensitive)

< 1 >

⚙

☐	Name ↗	Instance ID	Instance state ▼	Instance type ▼	Status check	Availability Zone ▼	Public IPv4 DNS
☐	backupballdatabase	i-0cda588ae91e8d177	Running  	t3.micro	 3/3 checks passed	us-east-1c	ec2-34-229-167-220.cc

```
# (1 unchanged attribute inlined)
```

```
}  
}
```

```
Plan: 0 to add, 0 to change, 1 to destroy.
```

Error: Instance cannot be destroyed

on Main.tf line 13:

```
13: resource "aws_instance" "backupballdatabase" {
```

Resource aws_instance.backupballdatabase has lifecycle.prevent_destroy set, but the plan calls for this resource to be destroyed. To avoid this error and continue with the plan, either disable lifecycle.prevent_destroy or reduce the scope of the plan using the -target option.

```
ubuntu@ip-172-31-15-233:~$
```

Step2: Now we can see the Terraform code, which has a lifecycle prevent destroy =true

```
Enter a value: yes

aws_instance.backupdballdatabase: Creating...
aws_instance.backupdballdatabase: Still creating... [00m10s elapsed]
aws_instance.backupdballdatabase: Creation complete after 13s [id=i-0cda588ae91e8d177]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
ubuntu@ip-172-31-15-233:~$ cat Main.tf
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "backupdballdatabase" {
  ami          = "ami-0c02fb55956c7d316"
  instance_type = "t3.micro"

  lifecycle {
    prevent_destroy = true
  }

  tags = {
    Name = "backupdballdatabase"
  }
}
```

```
ubuntu@ip-172-31-15-233:~$ ls
Main.tf  aws  awscli2.zip  terraform.tfstate  terraform.tfstate.backup
ubuntu@ip-172-31-15-233:~$ |
```

```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "backupdballdatabase" {
  ami          = "ami-0c02fb55956c7d316"
  instance_type = "t3.micro"

  lifecycle {
    prevent_destroy = true
  }

  tags = {
    Name = "backupdballdatabase"
  }
}

```

terraform init Initializes the Terraform project and downloads the required providers.

terraform fmt – Automatically formats Terraform files into the standard style.

terraform validate – Checks whether the Terraform configuration is valid.

terraform plan – Shows what Terraform will create, change, or delete before making changes.

terraform apply – Creates or updates the infrastructure.

terraform destroy – Deletes the infrastructure managed by Terraform.